

程序设计 II 第 3 周 题目汇总（完整版）

来源：Matrix·课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项

25 程设 II-周 3-课后 1

类型：编程题

题面

题目描述

请实现一个时间类 Time，用于表示一天中的时间（24 小时制）。要求如下：

1. **私有数据成员：**
 - hour, minute, second (均为 int 类型)。
2. **公有成员函数：**
 - void setTime(int h, int m, int s)：设置时间。
 - void addSeconds(int s)：增加给定的秒数。注意处理秒、分、时的进位（例如 59 秒加 2 秒变为 1 分 1 秒；23 点加进位可能变为 0 点）。
 - void printTime()：按照 hh:mm:ss 的格式输出时间。**注意：不足两位的数字需要前导补零（例如 08:05:09）。**

在 main 函数中，首先读取三个整数作为初始的时、分、秒，然后读取一个整数表示需要增加的秒数。输出增加秒数后的时间。

样例输入

23 59 50 15

样例输出

00:00:05

提交文件：Time.cpp

平台提供的初始文件

文件 Time.h：

```
#ifndef TIME_H
#define TIME_H
```

```
class Time {
```

```

private:
    int hour;
    int minute;
    int second;

public:
    // 设置时间
    void setTime(int h, int m, int s);

    // 增加秒数
    void addSeconds(int s);

    // 格式化输出时间
    void printTime() const;
};

#endif

```

文件 main.cpp :

```

#include <iostream>
#include "Time.h"

using namespace std;

int main() {
    int h, m, s, addS;
    while (cin >> h >> m >> s >> addS) {
        Time t;
        t.setTime(h, m, s);
        t.addSeconds(addS);
        t.printTime();
    }
    return 0;
}

```

25 程设 II-周 3-课后 2

类型：编程题

题面

题目描述

设计一个分数类 Fraction，实现分数的化简输出。要求如下：

1. **私有数据成员：**

- numerator (int 类型)：分子。

- denominator (int 类型)：分母。

2. 公有成员函数：

- void setFraction(int n, int d)：设置分子和分母。
- void simplify()：将分数化为最简分数（例如 4/8 化简为 1/2）。提示：你可以编写一个求最大公约数（GCD）的内部逻辑。
- void print()：按 分子/分母 的格式输出分数。如果分母化简后为 1，则只输出分子即可。

3. 化简原则：

- 利用最大公约数（GCD）进行约分。
- **符号规范化**：若分数为负，负号应统一保留在分子上（即分母始终保持为正）。若分子分母均为负，则结果为正。

4. 输入输出：从标准输入读取一对整数，保证分母不为 0。

在 main 函数中读取一对整数（分子和分母，保证分母不为 0），设置到对象中并化简，最后输出。

样例输入 1

-12 -18

样例输出 1

2/3

样例输入 2

10 -5

样例输出 2

-2

提交文件：Fraction.cpp

平台提供的初始文件

文件 Fraction.h：

```
#ifndef FRACTION_H
#define FRACTION_H
```

```
class Fraction {
private:
    int numerator;
    int denominator;
```

```
// 辅助私有函数：计算最大公约数(GCD)
```

```

    int gcd(int a, int b) const;

public:
    // 设置分子和分母
    void setFraction(int n, int d);

    // 化简分数
    void simplify();

    // 打印输出分数
    void print() const;
};

#endif

```

文件 main.cpp :

```

#include <iostream>
#include "Fraction.h"

using namespace std;

int main() {
    int n, d;
    while (cin >> n >> d) {
        Fraction f;
        f.setFraction(n, d);
        f.simplify();
        f.print();
    }
    return 0;
}

```

25 程设 II-周 3-课后 3

类型：编程题

题面

题目描述

请你完善一个简单的学生成绩管理系统，实现以下功能：

1. 存储学生的学号和三门课程的成绩。
2. 计算平均成绩。
3. 判断学生是否优秀
4. 输出学生平均成绩（四舍五入到一位小数）和优秀情况

优秀要求:每门课成绩大于等于 75，并且平均成绩大于等于 80

输入格式

输入的第一行包含一个整数 N，表示学生数量。

接下来 N 行，每行包含一个整数 id 和三个浮点数 s1, s2, s3，分别表示学生的学号和三门课程的成绩。

输出格式

输出 N 行，每行包含一个整数和一个浮点数，分别表示学生的学号、平均成绩和优秀情况，两者之间用一个空格分隔。平均成绩保留一位小数，优秀学生输出 excellent，否则输出 good。

样例输入 2 1001 80.0 85.0 90.0 1002 60.0 55.5 64.5

样例输出 1001 85.0 excellent 1002 60.0 good

提交文件：student.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "student.h"

using namespace std;

int main() {
    int N;
    cin >> N;

    for (int i = 0; i < N; i++) {
        int id;
        double s1, s2, s3;
        cin >> id >> s1 >> s2 >> s3;

        Student stu;
        stu.setStudent(id, s1, s2, s3);
        stu.printStatus();
    }

    return 0;
}
```

文件 student.h：

```
#ifndef STUDENT_H
#define STUDENT_H
```

```
class Student {
private:
    int id;
    double score1, score2, score3;

public:
    void setStudent(int id_, double s1, double s2, double s3);
    double getAverage() const;
    bool isExcellent() const;
    void printStatus() const;
};

#endif
```

25 程设 II-周 3-课后 4

类型：编程题

题面

问题描述

你需要模拟一个电梯系统的基本操作。电梯有当前楼层、目标楼层和状态三个成员变量。电梯状态为 0 表示停止，1 表示向上移动，-1 表示向下移动。初始时电梯静止。

输入格式

第一行包含两个整数 N 和 M ，其中 N 表示操作的次数， M 表示电梯所在的初始楼层。接下来 N 行，每行描述一个操作：

- 1 x ：根据 x (x 为整数) 更新电梯的目标楼层和状态。
 - 电梯静止时，设置目标楼层为 x ，并根据 x 与当前楼层的关系设置状态为向上、向下或保持静止。
 - 如果电梯处于向上（向下）状态时，只有当 x 大于（小于）当前目标楼层时，才更新目标楼层为 x 。
- 2：电梯向目标楼层移动一层；当电梯到达目标楼层时，电梯静止。
- 3：输出电梯所在的当前楼层。
- 4：输出电梯的状态（0、1 或 -1）。

输出格式

对于每个操作 3，输出一行包含当前楼层的整数；对于每个操作 4，输出一行包含电梯状态的整数。

样例输入

```
7 1
4
```

```
1 5
4
2
3
2
3
```

示例输出

```
0
1
2
3
```

提交文件：elevator.cpp

平台提供的初始文件

文件 elevator.h：

```
#ifndef ELEVATOR_H
#define ELEVATOR_H

class Elevator {
private:
    int currentFloor;
    int targetFloor;
    int state;

public:
    void init(int);
    void move();
    void goToFloor(int);
    int getCurrentFloor() const;
    int getState() const;
};

#endif
```

文件 main.cpp：

```
#include <iostream>
#include "elevator.h"
using namespace std;

int main() {
    Elevator e;
    int N, M;
    cin >> N >> M;
    e.init(M);
```

```

for (int i = 0; i < N; i++) {
    int op;
    cin >> op;
    if (op == 1) {
        int target;
        cin >> target;
        e.goToFloor(target);
    } else if (op == 2) {
        e.move();
    } else if (op == 3) {
        cout << e.getCurrentFloor() << endl;
    } else if (op == 4) {
        cout << e.getState() << endl;
    }
}

return 0;
}

```

25 程设 II-周 3-课堂 1

类型：编程题

题面

题目描述：实现 `Rectangle` 类

矩形对象 `Rectangle` 类包含 **长度(length)**、**宽度(width)**以及**绘制字符(style)** 三个变量。请根据类声明，实现该类中尚未定义的成员函数，使程序能够正确运行。

成员函数功能说明

1. **Set(int, int, char)**:同时设置矩形的长度、宽度以及绘制字符。(注意参数顺序，分别是长度、宽度和绘制字符！)
2. **getLength()**: 返回矩形的长度。
3. **getWidth()**: 返回矩形的宽度。
4. **getStyle()**: 返回矩形的绘制字符。
5. **getArea()**: 计算并返回矩形的面积。
6. **getPerimeter()**: 计算并返回矩形的周长。
7. **print()**: 按照矩形的长度和宽度，使用指定字符输出对应的矩形图案。(注：仅需输出矩形边，内部无需填充)

输入说明

输入一行，包含三个数据：length width style 分别表示矩形的长度、宽度和绘制字符。保证 `length > 1, width > 1`

输出说明

程序首先输出矩形的基本信息（长度、宽度、字符、面积、周长），然后输出对应的矩形图案。

样例输入

7 3 *

示例输出

```
Length: 7
Width: 3
Style: *
Area: 21
Perimeter: 20
*****
*   *
*****
```

提交文件：rectangle.cpp

平台提供的初始文件

文件 main.cpp：

```
#include "rectangle.h"
#include <iostream>

using namespace std;

int main(){
    Rectangle rect;
    int length, width;
    char style;

    cin>>length>>width>>style;

    rect.Set(length, width, style);

    rect.printInfo();
    rect.print();

    return 0;
}
```

文件 rectangle.h：

```
#ifndef RECTANGLE_H
#define RECTANGLE_H

#include <iostream>
```

```
using namespace std;

class Rectangle {
public:
    void Set(int length, int width, char style);
    int getLength() const;
    int getWidth() const;
    char getStyle() const;
    int getArea() const;
    int getPerimeter() const;
    void print() const;
    void printInfo() const{
        cout<<"Length: "<<getLength()<<endl;
        cout<<"Width: "<<getWidth()<<endl;
        cout<<"Style: "<<getStyle()<<endl;
        cout<<"Area: "<<getArea()<<endl;
        cout<<"Perimeter: "<<getPerimeter()<<endl;
    }

private:
    int width;
    int length;
    char style;
};

#endif
```

25 程设 II-周 3-课堂 2

类型：编程题

题面

题目描述

请设计一个简单的银行账户类 BankAccount，模拟基础的存取款操作。要求如下：

1. **私有数据成员：**
 - balance (int 类型)：账户余额。
2. **公有成员函数：**
 - void init(int initialBalance)：初始化账户余额。
 - void deposit(int amount)：存款操作，将金额累加到余额中。
 - void withdraw(int amount)：取款操作。如果余额充足，则扣除金额；如果余额不足以支付取款金额，则不进行扣除，并在屏幕上输出 Insufficient funds。
 - int getBalance()：返回当前余额。

在 main 函数中，依次读取三个整数：初始余额、存款金额、取款金额。依次调用上述函数进行操作，最后输出操作完成后的账户余额。

样例输入 1

100 50 200

样例输出 1

Insufficient funds 150

样例输入 2

100 100 50

样例输出 2

150

提交文件：BankAccount.cpp

平台提供的初始文件

文件 BankAccount.h：

```
#ifndef BANKACCOUNT_H
#define BANKACCOUNT_H

class BankAccount {
private:
    int balance;

public:
    // 初始化账户余额
    void init(int initialBalance);

    // 存款操作
    void deposit(int amount);

    // 取款操作
    void withdraw(int amount);

    // 获取当前余额
    int getBalance() const;
};

#endif
```

文件 main.cpp：

```
#include <iostream>
```

```
#include "BankAccount.h"

using namespace std;

int main() {
    int initBal, depAmt, withAmt;
    // 循环读取，支持Matrix 平台的多组数据测试
    while (cin >> initBal >> depAmt >> withAmt) {
        BankAccount account;
        account.init(initBal);
        account.deposit(depAmt);
        account.withdraw(withAmt);

        cout << account.getBalance();
    }
    return 0;
}
```

25 程设 II-周 3-理论

类型：选择题（20 题）

第 1 题

在程序设计中，ADT 的全称是什么？

- A. Advanced Data Technique
- B. Abstract Data Type
- C. Array Data Type
- D. Allocated Data Type

答案：B

解析：ADT 是 Abstract Data Type 的缩写，即抽象数据类型。

第 2 题

抽象数据类型（ADT）的数字化描述内容主要包括什么？

- A. 仅包含属性（关于事物数据特征）
- B. 仅包含方法（关于事物的行为与操作特征）
- C. 包含属性（数据特征）和方法（操作特征）
- D. 包含内存地址与指针操作

答案：C

解析：ADT 描述内容包括属性（关于事物数据特征）和方法（关于事物的行为与操作特征）。

第 3 题

在 C++ 中，class 关键字定义的类成员，如果没有显式指定访问说明符，默认访问权限是？

- A. public
- B. private
- C. protected
- D. static

答案：B

解析：在 C++ 中，class 默认权限是 private，而 struct 默认权限是 public。

第 4 题

在 C++ 中，struct 关键字定义的类成员，如果没有显式指定访问说明符，默认访问权限是？

- A. public
- B. private
- C. protected
- D. inline

答案：A

解析：struct 定义的类成员默认是公有的（public）。

第 5 题

关于 C 语言和 C++ 中的 struct，以下说法正确的是？

- A. C 语言的 struct 可以包含成员函数
- B. C++ 的 struct 只能包含成员变量，不能包含成员函数
- C. C++ 的 struct 既可以包含成员变量，也可以包含成员函数
- D. 两者的语法和语义没有任何区别

答案：C

解析：C 语言结构体只能包含变量；C++ 中的 struct 本质上是一种类，可以包含成员函数。

第 6 题

用于表示类成员仅对象内部可访问，对象使用者不可直接访问的关键字是？

- A. public
- B. private
- C. protected
- D. hidden

答案：B

解析：private 关键字用于实现封装中的信息隐藏。

第 7 题

在类的外部定义成员函数时，需要使用什么运算符来指明该函数属于哪个类？

- A. .
- B. ->
- C. ::
- D. :

答案：C

解析：:: 是作用域解析运算符，用于在类外实现成员函数。

第 8 题

在 C++ 中，如果将成员函数的实现直接写在类定义的 {} 内部，该函数会被视为？

- A. 外部函数
- B. 纯虚函数
- C. 内联函数 (inline)
- D. 静态函数

答案：C

解析：在类声明中定义的函数会被编译器处理为内联函数，以提高运行效率。

第 9 题

在成员函数声明的末尾加上 const 关键字（如 int getYear() const;），这表示什么含义？

- A. 该函数返回一个常量
- B. 该函数不能被外部调用
- C. 该函数不能修改类的数据成员
- D. 该函数的运行速度更快

答案：C

解析：const 成员函数承诺不修改对象的数据成员。

第 10 题

若有 DATE date1;，要调用 date1 的公有成员函数 Set，正确的语法是？

- A. date1->Set(2023, 10, 1);
- B. date1.Set(2023, 10, 1);
- C. DATE::Set(2023, 10, 1);

- D. Set(date1, 2023, 10, 1);

答案：B

解析：普通对象使用点运算符 . 来访问公有成员。

第 11 题

以下关于对象的描述，错误的是？

- A. 对象是类的实例
- B. 类定义的变量称为对象
- C. 类和对象是同一个概念，本质就是存有实际数据的模板
- D. 自动对象的默认初始化值为不确定

答案：C

解析：类是蓝图（不占数据空间），对象才是根据蓝图创建的实例。

第 12 题

假设 month 是 DATE 类的私有成员，以下哪种做法可以在类外部访问或修改它？

- A. 直接通过 date1.month = 10; 赋值
- B. 编写公有成员函数（如 getMonth() 和 Set()）供外部调用
- C. 使用 DATE::month 访问
- D. 无法在外部以任何方式获取其值

答案：B

解析：私有成员只能通过类提供的公有接口（Getter/Setter）来间接操作。

第 13 题

将数据与操作数据的函数绑定在一起，并隐藏内部实现细节的过程，这体现了面向对象编程的哪种特性？

- A. 继承
- B. 多态
- C. 封装
- D. 泛型

答案：C

解析：封装（Encapsulation）的作用就是将属性和行为结合，并控制访问权限。

第 14 题

在 C++ 类的常规设计中，类的声明（包含成员变量和函数原型）通常放在哪种文件中？

- A. .cpp 文件
- B. .hpp 或 .h 文件
- C. .txt 文件
- D. .exe 文件

答案：B

解析：通常声明放在头文件（.h 或 .hpp），实现放在源文件（.cpp）。

第 15 题

如果 DATE.cpp 中需要使用到 DATE.hpp 中定义的类声明，应该在该 .cpp 文件的开头添加什么？

- A. import DATE.hpp;
- B. using DATE.hpp;
- C. #include "DATE.hpp"
- D. #define DATE.hpp

答案：C

解析：使用 #include 预处理命令引入自定义头文件。

第 16 题

以下哪一项不是 C++ 中的类关键字（用于定义类的关键字）？

- A. class
- B. struct
- C. union
- D. object

答案：D

解析：C++ 的类关键字包括 class、struct 和 union；object 是对象，不是关键字。

第 17 题

void DATE::Increment() { day++; ... } 中的 DATE:: 的作用是什么？

- A. 声明这是一个新的数据类型
- B. 指明 Increment 是类 DATE 里的成员函数
- C. 定义类的私有数据成员
- D. 创建一个类的对象

答案：B

解析：这是作用域限定符，告诉编译器该函数属于哪个类。

第 18 题

“抽象”在人们认知事物中的主要意义是？

- A. 将事物变得更加复杂化和数据化
- B. 舍弃本质特征，保留表面现象进行描述
- C. 透过事物的现象获得它的本质，通过本质识别或区分事物
- D. 为事物分配计算机语言中的内存地址

答案：C

解析：抽象是过滤次要细节，抓取本质特征的过程。

第 19 题

关于客户端代码（Client code），描述正确的是？

- A. 它是指操作系统的内核运行代码
- B. 使用类的软件（即调用类对象及其公有方法的代码）称为客户端代码
- C. 它是指实现类内部实现细节的函数库代码
- D. 客户端代码可以直接无视权限访问类的私有数据成员

答案：B

解析：类是被使用的，使用该类的代码（如 main 函数所在的程序）就是该类的客户端。

第 20 题

观察代码：DATE date1, date2; 这里的 date1 和 date2 被称作什么？

- A. 类 (Class)
- B. 函数 (Function)
- C. 抽象数据类型 (ADT)
- D. 对象 (Object) 或实例 (Instance)

答案：D

解析：类定义的变量就是对象。
